Kubernetes — 30 Interview Questions (Senior / Staff Engineer Level)

> Calibrated for 7+ YOE. Covers architecture, networking, storage, security, scheduling, scaling, debugging, and real-world scenarios. Pure technical — no company-specific fluff.

Category Breakdown

#	Category	Questions
1–5	Architecture & Core Internals	Control plane, etcd, API server, kubelet
6–10	Workloads, Scheduling & Lifecycle	Pods, Deployments, StatefulSets, DaemonSets, scheduling
11–15	Networking	Services, Ingress, DNS, CNI, Network Policies
16–18	Storage	PV/PVC, StorageClasses, StatefulSet volumes
19–22	Security	RBAC, ServiceAccounts, Pod Security, Secrets
23–25	Scaling & Performance	HPA, VPA, Cluster Autoscaler, resource management
26–28	Observability & Debugging	Logs, probes, debugging CrashLoopBackOff
29–30	Real-World Scenarios	Production troubleshooting, zero-downtime deployments

---

Q1: Walk me through what happens when you run `kubectl apply -f deployment.yaml` — from the moment you hit Enter to the container running.

Answer:

```
Step 1: kubectl CLIENT
  - kubectl reads deployment.yaml, validates against the OpenAPI schema
  - Authenticates with the API server (using kubeconfig: certificate/token)
  - Sends an HTTP POST/PATCH to the API server:
      POST https://<api-server>:6443/apis/apps/v1/namespaces/default/deployments

Step 2: API SERVER (kube-apiserver)
  - Receives the request
```

```
  - Authentication: verifies client identity (certificate, token, OIDC)
  - Authorization: checks RBAC — does this user/SA have permission to create
Deployments?
  - Admission Controllers: runs mutating webhooks (e.g., inject sidecar, add
labels)
    then validating webhooks (e.g., enforce policies like "no :latest tag")
  - Persists the Deployment object to etcd
  - Returns 201 Created to kubectl

Step 3: CONTROLLER MANAGER (kube-controller-manager)
  - Deployment Controller watches for new/changed Deployments
  - Sees new Deployment → creates a ReplicaSet object
  - ReplicaSet Controller watches for new ReplicaSets
  - Sees new ReplicaSet → creates Pod objects (desired replicas)
  - Pods are created with status: Pending (no node assigned yet)

Step 4: SCHEDULER (kube-scheduler)
  - Watches for Pods with no nodeName (Pending)
  - Runs filtering: eliminates nodes that can't run the Pod
    (insufficient CPU/memory, taints, affinity rules, etc.)
  - Runs scoring: ranks remaining nodes
    (spread, resource balance, affinity preferences)
  - Binds the Pod to the highest-scoring node:
    writes nodeName to the Pod object in etcd

Step 5: KUBELET (on the chosen node)
  - Watches API server for Pods assigned to its node
  - Sees new Pod → pulls container image (via container runtime: containerd/CRI-
O)
  - Creates the container(s) via CRI (Container Runtime Interface)
  - Sets up networking via CNI plugin (assigns Pod IP, sets up veth pairs)
  - Sets up volumes (mounts PVCs, ConfigMaps, Secrets)
  - Starts the container
  - Reports Pod status back to API server: Running

Step 6: KUBE-PROXY (on every node)
  - If a Service exists for this Deployment, kube-proxy updates
    iptables/IPVS rules so traffic to the Service ClusterIP
    gets forwarded to the new Pod's IP
```

What to highlight: The entire system is **declarative and watch-based**. No component calls another directly. They all watch etcd (via the API server) and react to state changes. This is the reconciliation loop pattern.

Q2: What is etcd's role in Kubernetes, and what happens if etcd goes down?

Answer:

Role: etcd is the **single source of truth** for the entire cluster. Every object (Pods, Deployments, Services, ConfigMaps, Secrets, RBAC rules) is stored in etcd as key-value pairs. The API server is the only component that talks to etcd directly.

What's stored:

```
/registry/pods/default/my-pod-abc123          → Pod spec + status
/registry/deployments/default/my-app          → Deployment spec
/registry/services/default/my-service         → Service definition
/registry/secrets/default/db-credentials      → Encrypted secret data
/registry/leases/kube-system/kube-scheduler   → Leader election lease
```

If etcd goes down:

```
Phase 1: API SERVER becomes read-only (from its cache) then fails
  - New creates/updates/deletes fail immediately
  - Existing running Pods continue running (kubelet doesn't need etcd)

Phase 2: CONTROLLERS stop reconciling
  - No new ReplicaSets, no scaling, no rescheduling
  - If a node dies during etcd outage, Pods aren't rescheduled

Phase 3: SCHEDULER stops scheduling
  - New Pods stay in Pending forever

Phase 4: EXISTING WORKLOADS keep running
  - Containers already on nodes continue (kubelet runs independently)
  - But no new changes, no healing, no scaling

Recovery:
  - Restore etcd from snapshot: etcdctl snapshot restore
  - Or bring etcd quorum back (if multi-node etcd cluster with 3/5 nodes)
```

Production setup: Always run etcd as a 3 or 5 node cluster (Raft consensus needs majority quorum: 2/3 or 3/5). Take automated snapshots. Separate etcd from worker nodes (dedicated hardware or VMs with fast SSDs — etcd is latency-sensitive).

Q3: Explain the difference between a Deployment, StatefulSet, and DaemonSet. When do you pick each?

Answer:

	Deployment	StatefulSet	DaemonSet
Identity	Pods are interchangeable (random names like `app-7b4f9c`)	Pods have stable, ordered identity (`db-0`, `db-1`, `db-2`)	One Pod per node
Scaling	Scale up/down freely, any order	Scale up sequentially (0→1→2), scale down reverse (2→1→0)	Automatically one per node (or per matching node)
Storage	Shared PVC or no persistent storage	Each Pod gets its own PVC (volumeClaimTemplates) — PVC survives Pod deletion	Usually hostPath or no storage
Network	No stable hostname per Pod	Stable DNS: `db-0.db-headless.namespace.svc.cluster.local`	N/A
Use case	Stateless apps (API servers, web frontends)	Stateful apps (databases, Kafka, ZooKeeper, Redis cluster)	Node-level agents (log collectors, monitoring, CNI, kube-proxy)

When to pick:

- **Deployment:** Default choice. Use for anything that doesn't need stable identity or per-Pod storage.
- **StatefulSet:** When Pods need stable network identity AND/OR their own persistent volume. Think databases, message brokers.
- **DaemonSet:** When you need exactly one Pod per node (or per qualifying node). Think Fluentd, Datadog agent, kube-proxy.

Q4: What are Admission Controllers? Explain mutating vs validating webhooks with an example.

Answer:

Admission controllers intercept requests to the API server **after authentication and authorization but before persistence to etcd**. They can modify or reject the request.

```
Request flow:
  kubectl apply → API Server → AuthN → AuthZ → Mutating Admission → Validating
Admission → etcd
```

```
Two types:

1. MUTATING ADMISSION WEBHOOK
  - Can MODIFY the object before it's saved
  - Runs FIRST
  - Example: Istio sidecar injector
    → You submit a Pod spec with 1 container
    → Mutating webhook adds the Envoy sidecar container automatically
    → Pod saved to etcd has 2 containers

    Before mutation:
      containers:
        - name: my-app
          image: my-app:v1

    After mutation (by Istio webhook):
      containers:
        - name: my-app
          image: my-app:v1
        - name: istio-proxy         ← injected
          image: istio/proxyv2:1.20 ← injected

2. VALIDATING ADMISSION WEBHOOK
  - Can only ACCEPT or REJECT — cannot modify
  - Runs AFTER mutating webhooks
  - Example: Policy enforcement (OPA Gatekeeper / Kyverno)
    → Reject Pods that use :latest tag
    → Reject Pods without resource limits
    → Reject Pods that run as root

    Policy: "All Pods must have CPU/memory limits"
    Pod spec has no limits → webhook returns: DENIED
    "Pod my-app rejected: missing resource limits"
```

Order: Mutating runs first (so policies validate the final, mutated object).

Q5: What is the difference between a ReplicaSet and a Deployment? Why do we almost never create ReplicaSets directly?

Answer:

A **ReplicaSet** ensures N identical Pods are running. It's the *mechanism* that maintains the desired replica count.

A **Deployment** is a *higher-level controller* that manages ReplicaSets. It adds:

- **Rolling updates** — creates a new ReplicaSet, scales it up, scales old one down
- **Rollback** — `kubectl rollout undo` switches back to the previous ReplicaSet
- **Revision history** — keeps old ReplicaSets (configurable: `revisionHistoryLimit`)
- **Update strategy** — `RollingUpdate` (default) or `Recreate`

```
Deployment: my-app (replicas: 3)
  └── ReplicaSet: my-app-7b4f9c (current, 3 Pods running)
  └── ReplicaSet: my-app-5a2d8e (previous revision, 0 Pods, kept for rollback)

On image update (v1 → v2):
  └── ReplicaSet: my-app-7b4f9c (v1, scaling DOWN: 3 → 2 → 1 → 0)
  └── ReplicaSet: my-app-c3e7f1 (v2, scaling UP: 0 → 1 → 2 → 3)
```

Why not create ReplicaSets directly? You lose rolling updates, rollbacks, and revision history. There's no mechanism to transition from one version to another — you'd have to manually create a new ReplicaSet and delete the old one. Deployments automate all of that.

Q6: Explain the Kubernetes scheduling process. What are taints, tolerations, and node affinity?

Answer:

Scheduling Flow

```
Pod created (no nodeName) → Scheduler picks it up

Phase 1: FILTERING (eliminate unsuitable nodes)
  - Does the node have enough CPU/memory?
  - Does the Pod tolerate the node's taints?
  - Does the Pod's nodeSelector/nodeAffinity match?
  - Is the node cordoned (unschedulable)?
  - PV affinity: does the PVC's zone match the node?

Phase 2: SCORING (rank remaining nodes)
  - LeastRequestedPriority: prefer nodes with more free resources
  - BalancedResourceAllocation: prefer even CPU/memory ratio
  - InterPodAffinity: prefer nodes that satisfy Pod affinity rules
  - SpreadConstraints: spread across zones/nodes

Phase 3: BIND
```

```
  - Pick highest-scoring node
  - Write nodeName to Pod spec
```

Taints & Tolerations

Taints are on nodes — they REPEL Pods that don't tolerate them.
Tolerations are on Pods — they ALLOW the Pod to be scheduled on tainted nodes.

```
Node taint (applied by admin):
  kubectl taint nodes node1 gpu=true:NoSchedule

  Effect: No Pod will be scheduled on node1 unless it tolerates this taint.

Pod toleration:
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

  This Pod CAN be scheduled on node1.

Common effects:
  NoSchedule     → Don't schedule new Pods (existing stay)
  PreferNoSchedule → Try not to, but allow if no other option
  NoExecute      → Evict existing Pods that don't tolerate
```

Use case: GPU nodes, dedicated node pools, cordoning nodes for maintenance.

Node Affinity

Like `nodeSelector` but more expressive — supports soft (preferred) and hard (required) rules.

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:   ← HARD rule
      nodeSelectorTerms:
        - matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values: ["us-east-1a", "us-east-1b"]

    preferredDuringSchedulingIgnoredDuringExecution:  ← SOFT rule
      - weight: 80
        preference:
```

```
          matchExpressions:
            - key: instance-type
              operator: In
              values: ["m5.xlarge"]
```

Hard: Pod MUST go to us-east-1a or 1b. Fails if no matching node.
Soft: Prefer m5.xlarge instances, but schedule elsewhere if unavailable.

Q7: What's the difference between a liveness probe, readiness probe, and startup probe?

Answer:

Probe	Purpose	If it FAILS	When to use
Liveness	Is the container alive and healthy?	kubelet **kills and restarts** the container	Detect deadlocks, hung processes
Readiness	Can the container accept traffic?	Pod removed from **Service endpoints** (no traffic routed)	Warming up, loading data, temporary overload
Startup	Has the container finished starting?	kubelet **kills and restarts** (after failureThreshold)	Slow-starting apps (Java, large ML models)

```
Key behavior:

Liveness:
  Container is running but deadlocked (thread stuck, infinite loop)
  Liveness probe fails → kubelet restarts the container
  Without liveness: container stays "Running" but serves nothing

Readiness:
  Container is starting up, loading a model into memory (takes 30s)
  Readiness probe fails → kube-proxy removes Pod from Service endpoints
  Traffic goes to other ready Pods
  Once ready → Pod added back to endpoints
  Without readiness: traffic hits Pod before it can serve → errors

Startup:
  Java app takes 120s to start (Spring Boot with large context)
  Without startup probe: liveness probe fires at 10s, container isn't ready
    → liveness fails → kubelet restarts → restart loop
  With startup probe: liveness/readiness probes disabled until startup succeeds
    → gives the app time to boot
    → once startup passes, liveness and readiness take over
```

```
Probe types:
  httpGet:    GET /healthz → expect 200
  tcpSocket:  connect to port 8080 → expect connection success
  exec:       run command inside container → expect exit code 0
```

Best practice at 7 YOE: ALWAYS set readiness probes. Set liveness probes for long-running processes that can deadlock. Use startup probes for slow-starting apps with aggressive liveness settings.

Q8: Explain Kubernetes networking. How does Pod-to-Pod communication work across nodes?

Answer:

The Three Networking Rules in K8s

```
Rule 1: Every Pod gets its own unique IP address
Rule 2: Any Pod can communicate with any other Pod using its IP
         (without NAT) — even across nodes
Rule 3: Agents on a node (kubelet, kube-proxy) can communicate
         with all Pods on that node
```

How It Works (CNI Plugin Mechanics)

```
Same Node Communication:
  Pod A (10.244.1.5) → Pod B (10.244.1.8)

  Phase 1: Pod A sends packet to its virtual ethernet (veth) interface
  Phase 2: veth connects to the node's bridge (cbr0 / cni0)
  Phase 3: Bridge sees destination 10.244.1.8 is local
  Phase 4: Forwards to Pod B's veth interface
  Phase 5: Pod B receives the packet

  Path: Pod A → veth → bridge → veth → Pod B

Cross-Node Communication:
  Pod A (10.244.1.5 on Node 1) → Pod C (10.244.2.9 on Node 2)

  Phase 1: Pod A sends packet to its veth → node's bridge
  Phase 2: Bridge doesn't know 10.244.2.0/24 locally
  Phase 3: Packet goes to the node's routing table
  Phase 4: Route says 10.244.2.0/24 → Node 2 (via overlay or direct routing)
  Phase 5: Packet encapsulated (VXLAN/Geneve for overlay CNIs) or routed directly
```

```
  Phase 6: Arrives at Node 2, decapsulated
  Phase 7: Node 2's bridge routes to Pod C's veth
  Phase 8: Pod C receives the packet

Popular CNI plugins:
  Calico:  Uses BGP for direct routing (no overlay) — fastest
  Flannel: VXLAN overlay — simple, moderate performance
  Cilium:  eBPF-based — high performance, advanced features (security,
observability)
  Weave:   Mesh overlay — simple setup
```

Q9: What are Services in Kubernetes? Explain ClusterIP, NodePort, LoadBalancer, and Headless.

Answer:

```
ClusterIP (default):
  - Internal-only virtual IP
  - Accessible only from within the cluster
  - kube-proxy creates iptables/IPVS rules to load-balance across Pod IPs
  - Example: backend API service called by frontend Pods

  Service: my-api → ClusterIP 10.96.45.12:80
    → Pod 10.244.1.5:8080
    → Pod 10.244.2.8:8080
    → Pod 10.244.3.3:8080

NodePort:
  - Extends ClusterIP
  - Opens a static port (30000-32767) on EVERY node
  - External traffic can hit <any-node-IP>:<nodePort>
  - kube-proxy forwards to the ClusterIP → Pods

  Service: my-api → NodePort 31080
    External: http://192.168.1.10:31080 → Pod

LoadBalancer:
  - Extends NodePort
  - Provisions a cloud load balancer (AWS ELB/ALB, GCP LB)
  - Cloud LB gets a public IP → routes to NodePort → Pods
  - Used for production external traffic

  Service: my-api → LoadBalancer
```

```
    Public: http://a1b2c3.elb.amazonaws.com → NodePort → Pod

Headless (ClusterIP: None):
  - No virtual IP, no load balancing by kube-proxy
  - DNS returns the individual Pod IPs directly
  - Used with StatefulSets for stable per-Pod DNS

  DNS query: db-headless.default.svc.cluster.local
    → returns: 10.244.1.5, 10.244.2.8, 10.244.3.3 (all Pod IPs)

  Each Pod also gets:
    db-0.db-headless.default.svc.cluster.local → 10.244.1.5
    db-1.db-headless.default.svc.cluster.local → 10.244.2.8
```

When to use which:

- **ClusterIP:** Internal service-to-service communication (90% of cases)
- **NodePort:** Quick external access for testing/dev (not production)
- **LoadBalancer:** Production external traffic with cloud provider
- **Headless:** StatefulSets, service discovery where clients need individual Pod IPs

---

Q10: What are Network Policies and how do they work?

Answer:

Network Policies are Kubernetes-native **firewall rules** at the Pod level. By default, all Pods can talk to all other Pods (open network). Network Policies restrict this.

```
Default: All Pods can communicate freely (no policies)

When you apply a NetworkPolicy to a Pod:
  - ALL ingress/egress traffic to that Pod is DENIED by default
  - Only traffic explicitly allowed by the policy is permitted

Example: Allow only frontend Pods to reach backend Pods on port 8080

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:                        ← apply to Pods with label app=backend
    matchLabels:
      app: backend
  policyTypes:
```

```
    - Ingress
  ingress:
    - from:
        - podSelector:                  ← allow traffic FROM Pods with
app=frontend
            matchLabels:
              app: frontend
      ports:
        - port: 8080                    ← only on port 8080

Result:
  frontend Pod → backend Pod:8080    ✅ ALLOWED
  random Pod → backend Pod:8080      ❌ DENIED
  frontend Pod → backend Pod:9090    ❌ DENIED (wrong port)
```

Important: Network Policies require a **CNI plugin that supports them** (Calico, Cilium, Weave). Flannel does NOT enforce Network Policies.

Q11: How does DNS work inside a Kubernetes cluster?

Answer:

```
CoreDNS runs as a Deployment in kube-system namespace (typically 2 replicas).
Every Pod gets /etc/resolv.conf pointing to the CoreDNS Service ClusterIP.

DNS resolution patterns:

  Service (same namespace):
    my-service → my-service.default.svc.cluster.local
    Resolves to: ClusterIP (e.g., 10.96.45.12)

  Service (cross-namespace):
    my-service.other-namespace → my-service.other-namespace.svc.cluster.local

  Headless Service Pod:
    pod-0.my-headless.default.svc.cluster.local → Pod IP directly

  External:
    google.com → forwarded to upstream DNS (e.g., 8.8.8.8)

Search domains in Pod's /etc/resolv.conf:
  search default.svc.cluster.local svc.cluster.local cluster.local
  nameserver 10.96.0.10  ← CoreDNS Service IP
```

```
This is why you can just write "my-service" in code instead of the FQDN —
the search domains expand it automatically.
```

Common issue: DNS resolution latency under load. CoreDNS has limited resources by default. At scale (1000+ Pods doing frequent lookups), you may need to scale CoreDNS replicas or enable NodeLocal DNSCache (DaemonSet that caches DNS on every node).

Q12: What is an Ingress? How is it different from a LoadBalancer Service?

Answer:

```
LoadBalancer Service:
  - One cloud load balancer PER service
  - 10 services = 10 load balancers = 10 public IPs = expensive
  - No path-based or host-based routing
  - L4 (TCP/UDP level)

Ingress:
  - One load balancer for ALL services (via Ingress Controller)
  - Routes traffic based on HOST and PATH
  - L7 (HTTP level) — can inspect headers, paths, hosts
  - TLS termination, rewrites, rate limiting (depending on controller)

Example:
  One Ingress Controller (NGINX / Traefik / ALB)
    → api.myapp.com/users   → user-service:8080
    → api.myapp.com/orders  → order-service:8080
    → web.myapp.com/        → frontend-service:3000

  Instead of 3 LoadBalancer Services (3 cloud LBs, 3 public IPs),
  you have 1 Ingress Controller + 1 cloud LB.

Ingress spec:
  rules:
    - host: api.myapp.com
      http:
        paths:
          - path: /users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
```

```
                number: 8080
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 8080
```

Key point: Ingress by itself does nothing. You need an **Ingress Controller** (NGINX Ingress Controller, Traefik, AWS ALB Controller, etc.) installed in the cluster. The Ingress resource is just the routing rule; the controller is the engine that implements it.

Q13: Explain Persistent Volumes (PV), Persistent Volume Claims (PVC), and StorageClasses.

Answer:

```
PV (Persistent Volume):
  - A piece of storage provisioned in the cluster
  - Can be an EBS volume, NFS share, GCE Persistent Disk, etc.
  - Cluster-level resource (not namespaced)
  - Has a lifecycle independent of any Pod

PVC (Persistent Volume Claim):
  - A REQUEST for storage by a user/Pod
  - Specifies size, access mode, storage class
  - Namespaced resource
  - Kubernetes BINDS a PVC to a matching PV

StorageClass:
  - Defines HOW to dynamically provision storage
  - Specifies the provisioner (aws-ebs, gce-pd, etc.), parameters, reclaim policy
  - Eliminates the need to pre-create PVs manually

Flow:

  STATIC PROVISIONING (admin pre-creates PVs):
    Admin creates PV (10Gi, ReadWriteOnce, EBS vol-123)
    User creates PVC requesting 10Gi, ReadWriteOnce
    Kubernetes binds PVC → PV
    Pod mounts the PVC

  DYNAMIC PROVISIONING (with StorageClass):
```

```
    Admin creates StorageClass (provisioner: aws-ebs, type: gp3)
    User creates PVC requesting 10Gi with storageClassName: gp3
    Kubernetes calls the provisioner → creates EBS volume automatically
    PV is auto-created, bound to PVC
    Pod mounts the PVC

Reclaim policies:
  Retain:  PV kept after PVC deletion (manual cleanup)
  Delete:  PV and underlying storage deleted when PVC is deleted
  Recycle: (deprecated) basic wipe and reuse

Access modes:
  ReadWriteOnce (RWO):  one node can mount read-write
  ReadOnlyMany  (ROX):  many nodes can mount read-only
  ReadWriteMany (RWX):  many nodes can mount read-write (NFS, EFS)
```

Q14: How do StatefulSets handle storage differently from Deployments?

Answer:

```
Deployment:
  - All Pods share the same PVC (or have no persistent storage)
  - If Pod dies and reschedules to another node, it gets the same shared PVC
  - No per-Pod storage identity

StatefulSet:
  - Uses volumeClaimTemplates — each Pod gets its OWN PVC
  - PVCs are named deterministically: data-db-0, data-db-1, data-db-2
  - If db-1 Pod dies, a new db-1 Pod is created and re-attached to data-db-1
  - PVCs are NOT deleted when Pods are deleted (data survives)

Example:
  StatefulSet: db (replicas: 3)
    volumeClaimTemplates:
      - metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
```

```
  Creates:
    Pod db-0 → PVC data-db-0 → PV (50Gi EBS)
    Pod db-1 → PVC data-db-1 → PV (50Gi EBS)
    Pod db-2 → PVC data-db-2 → PV (50Gi EBS)

  db-1 crashes → new db-1 Pod → re-mounts data-db-1 (same data)

  Scale down to 2 replicas:
    Pod db-2 deleted
    PVC data-db-2 STILL exists (not deleted)
    Scale back to 3 → new db-2 re-mounts data-db-2

This is why StatefulSets are used for databases — each replica has
its own persistent identity and storage.
```

Q15: What is RBAC in Kubernetes? Explain Role, ClusterRole, RoleBinding, ClusterRoleBinding.

Answer:

```
RBAC = Role-Based Access Control
Controls WHO can do WHAT on WHICH resources.

4 objects:

Role              → defines permissions within a NAMESPACE
ClusterRole       → defines permissions CLUSTER-WIDE
RoleBinding       → grants a Role to a user/SA within a NAMESPACE
ClusterRoleBinding → grants a ClusterRole CLUSTER-WIDE

Example: Allow CI/CD service account to manage Deployments in "staging" namespace

Step 1: Create a Role
  apiVersion: rbac.authorization.k8s.io/v1
  kind: Role
  metadata:
    namespace: staging
    name: deployment-manager
  rules:
    - apiGroups: ["apps"]
      resources: ["deployments"]
      verbs: ["get", "list", "create", "update", "patch", "delete"]
```

```
Step 2: Create a RoleBinding
  kind: RoleBinding
  metadata:
    namespace: staging
    name: ci-deploy-binding
  subjects:
    - kind: ServiceAccount
      name: ci-bot
      namespace: staging
  roleRef:
    kind: Role
    name: deployment-manager

Result:
  ci-bot SA can CRUD Deployments in staging namespace ONLY.
  ci-bot SA CANNOT touch Deployments in production namespace.
  ci-bot SA CANNOT touch Pods, Services, Secrets (not in the Role).

ClusterRole + ClusterRoleBinding:
  Same idea but applies across ALL namespaces.
  Use for: cluster admins, node management, PV management.
```

Q16: How do you manage Secrets in Kubernetes? What are the security concerns?

Answer:

```
Default Kubernetes Secrets:
  - Stored in etcd as base64 encoded (NOT encrypted by default!)
  - base64 is encoding, NOT encryption — anyone with etcd access can read them
  - Mounted as files in /var/run/secrets/... or exposed as env vars

Security concerns:
  1. etcd stores Secrets in plaintext (unless encryption at rest is configured)
  2. Any Pod in a namespace can mount any Secret in that namespace (unless RBAC
restricts it)
  3. Secrets in env vars appear in process listings and crash dumps
  4. Secrets are cached on nodes in tmpfs — accessible to anyone with node SSH
access

Hardening steps:

  Level 1: Enable encryption at rest for etcd
```

```
    apiVersion: apiserver.config.k8s.io/v1
    kind: EncryptionConfiguration
    resources:
      - resources: ["secrets"]
        providers:
          - aescbc:
              keys:
                - name: key1
                  secret: <base64-encoded-key>

  Level 2: Use RBAC to restrict who can read Secrets
    Only specific ServiceAccounts should have "get" on Secrets

  Level 3: Use external secret managers
    - HashiCorp Vault (with Vault Agent Injector)
    - AWS Secrets Manager (with External Secrets Operator)
    - Azure Key Vault
    Secrets never stored in etcd — fetched at runtime from external vault

  Level 4: Mount as files, not env vars
    Files can have restricted permissions (0400)
    Env vars leak in process listings and pod describe output

  Level 5: Enable audit logging for Secret access
    Track who accessed which Secrets and when
```

Q17: What is a Pod Security context? What is a PodSecurityAdmission / PodSecurityStandard?

Answer:

```
Security Context: Per-Pod or per-container security settings

  securityContext:
    runAsNonRoot: true          ← container must run as non-root user
    runAsUser: 1000             ← run as UID 1000
    readOnlyRootFilesystem: true ← container filesystem is read-only
    allowPrivilegeEscalation: false
    capabilities:
      drop: ["ALL"]             ← drop all Linux capabilities

Pod Security Standards (PSS) — 3 levels:
```

```
  Privileged:  No restrictions (for system-level Pods like CNI, kube-proxy)
  Baseline:    Prevents obvious escalations (no hostNetwork, no privileged, etc.)
  Restricted:  Hardened (must run as non-root, drop caps, read-only fs)

Pod Security Admission (PSA) — enforces PSS at namespace level:

  kubectl label namespace production \
    pod-security.kubernetes.io/enforce=restricted \
    pod-security.kubernetes.io/warn=restricted

  Any Pod in "production" namespace that violates "restricted" policy → REJECTED.

This replaced the deprecated PodSecurityPolicy (PSP) in K8s 1.25+.
```

Q18: Explain Horizontal Pod Autoscaler (HPA). How does it work and what are its limitations?

Answer:

```
HPA automatically scales Pod replicas based on observed metrics.

How it works:

  Phase 1: METRICS COLLECTION
    - Metrics Server collects CPU/memory usage from kubelet (every 15s default)
    - Custom metrics can come from Prometheus Adapter

  Phase 2: HPA CONTROLLER checks every 15 seconds (configurable)
    - Reads current metric value (e.g., avg CPU = 80%)
    - Compares to target (e.g., target CPU = 50%)
    - Calculates desired replicas:
        desiredReplicas = currentReplicas × (currentMetric / targetMetric)
        desiredReplicas = 3 × (80 / 50) = 4.8 → rounds up to 5

  Phase 3: SCALE
    - HPA updates Deployment replicas: 3 → 5
    - Deployment controller creates 2 new Pods
    - Once new Pods are ready and absorb load, CPU drops

  Phase 4: COOLDOWN
    - Scale-up cooldown: 0s (immediate, default since K8s 1.18)
    - Scale-down cooldown: 5 minutes (default stabilization window)
    - Prevents flapping (rapid scale up/down cycles)
```

```
Example:
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  spec:
    scaleTargetRef:
      kind: Deployment
      name: my-api
    minReplicas: 2
    maxReplicas: 20
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 50

Limitations:
  - Requires resource requests to be set (HPA uses requests as baseline)
  - Metrics Server latency: ~15-30s delay before HPA reacts
  - Doesn't handle burst traffic well (scale-up takes 30-60s for new Pods)
  - Custom metrics (RPS, queue depth) need Prometheus + Adapter setup
  - Can conflict with Cluster Autoscaler (HPA wants more Pods but nodes are full)
  - VPA and HPA on the same metric (CPU) can conflict — don't overlap
```

Q19: What is the Cluster Autoscaler? How does it work with HPA?

Answer:

```
Cluster Autoscaler scales the NUMBER OF NODES (not Pods).
HPA scales Pods. They work together:

  Load increases → HPA creates more Pods
  → Not enough node capacity for new Pods → Pods stay Pending
  → Cluster Autoscaler detects Pending Pods
  → Provisions new nodes (calls cloud API: EC2, GKE node pool, etc.)
  → New nodes join cluster → Pending Pods get scheduled

  Load decreases → HPA scales down Pods
  → Some nodes become underutilized (<50% of capacity)
  → Cluster Autoscaler waits stabilization period (~10 min)
```

```
  → Evicts Pods (respects PDBs) → drains node → terminates node

Scale-up flow:
  Phase 1: DETECT Pending Pods — scheduler can't place them (no resources)
  Phase 2: SIMULATE — which node group/pool can fit these Pods?
  Phase 3: PROVISION — call cloud API to add node(s)
  Phase 4: WAIT — node boots, joins cluster, passes health check (~1-3 min)
  Phase 5: SCHEDULE — pending Pods are assigned to new node

Scale-down flow:
  Phase 1: IDENTIFY underutilized nodes (CPU+memory < threshold, default 50%)
  Phase 2: CHECK — are there Pods that can't be moved?
    (PDBs, local storage, node-selector-only Pods)
  Phase 3: DRAIN — evict Pods (respecting PDBs and graceful termination)
  Phase 4: TERMINATE — deregister and delete the node

Key gotcha:
  Scale-up takes 1-3 minutes (node provisioning). For latency-sensitive
  workloads, over-provision slightly (keep warm spare capacity) to avoid
  waiting for new nodes during traffic spikes.
```

Q20: How do you do a zero-downtime deployment in Kubernetes?

Answer:

```
Required ingredients:

1. ROLLING UPDATE STRATEGY (default in Deployments)
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0     ← never kill old Pods before new ones are ready
      maxSurge: 1           ← allow 1 extra Pod during rollout

  Flow:
    Start:   v1-pod-a ✅, v1-pod-b ✅, v1-pod-c ✅   (3 replicas)
    Step 1:  v2-pod-d created (surge), v1 all still running    (4 Pods)
    Step 2:  v2-pod-d passes readiness → v1-pod-a terminated   (3 Pods)
    Step 3:  v2-pod-e created, passes readiness → v1-pod-b terminated
    Step 4:  v2-pod-f created, passes readiness → v1-pod-c terminated
    End:     v2-pod-d ✅, v2-pod-e ✅, v2-pod-f ✅
```

```
2. READINESS PROBE (critical!)
  - New Pod must pass readiness BEFORE old Pod is terminated
  - Without readiness: K8s assumes Pod is ready immediately
    → kills old Pod → new Pod isn't actually ready → dropped requests

3. GRACEFUL SHUTDOWN
  In your application:
    - Handle SIGTERM signal
    - Stop accepting new requests
    - Finish in-flight requests
    - Exit cleanly

  In Pod spec:
    terminationGracePeriodSeconds: 30  ← K8s waits 30s before SIGKILL

4. PRE-STOP HOOK (handles the race condition)
  Problem: kube-proxy takes a few seconds to update iptables rules.
    Old Pod gets SIGTERM and starts shutting down, but kube-proxy
    still routes traffic to it for a brief window → errors.

  Fix:
    lifecycle:
      preStop:
        exec:
          command: ["sleep", "5"]  ← wait 5s before shutdown

  Timeline:
    T+0s: Pod marked for termination
    T+0s: kube-proxy starts removing Pod from endpoints (async)
    T+0-5s: preStop hook runs (sleep 5), Pod still accepts traffic
    T+5s: kube-proxy has updated, no new traffic arrives
    T+5s: SIGTERM sent, app gracefully shuts down
    T+35s: SIGKILL if still running

5. POD DISRUPTION BUDGET (PDB)
  Prevents K8s from killing too many Pods at once during voluntary disruptions
  (rolling updates, node drain, cluster autoscaler scale-down):

  apiVersion: policy/v1
  kind: PodDisruptionBudget
  spec:
    minAvailable: 2         ← always keep at least 2 Pods running
    selector:
      matchLabels:
        app: my-api
```

Q21: What is a ConfigMap and how does it differ from a Secret?

Answer:

```
ConfigMap: stores non-sensitive configuration as key-value pairs
Secret:    stores sensitive data (passwords, tokens, keys)

Key differences:
  - Secrets are base64 encoded (ConfigMaps are plaintext)
  - Secrets can have encryption at rest enabled in etcd
  - Secrets have type field (Opaque, kubernetes.io/tls, etc.)
  - kubelet keeps Secrets in tmpfs (not written to disk on nodes)

Mounting options (same for both):

  As environment variables:
    env:
      - name: DB_HOST
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: database_host

  As volume files:
    volumes:
      - name: config
        configMap:
          name: app-config
    volumeMounts:
      - name: config
        mountPath: /etc/config

  Hot reload: When mounted as files, updates to ConfigMap/Secret
  propagate to the Pod automatically (~30-60s, kubelet sync period).
  When used as env vars: NO hot reload — Pod must restart.
```

Q22: What happens when a node becomes NotReady? Walk through the Pod eviction process.

Answer:

```
Phase 1: DETECTION
  - kubelet sends heartbeats to API server every 10s (node lease)
  - If API server doesn't receive heartbeat for 40s (default):
      node-monitor-grace-period = 40s
  - Node status changes: Ready → NotReady

Phase 2: TAINT APPLIED
  - Node controller adds taint:
      node.kubernetes.io/not-ready:NoExecute
  - Also: node.kubernetes.io/unreachable:NoExecute (if network partition)

Phase 3: TOLERATION TIMEOUT
  - Pods have a default toleration for not-ready with
    tolerationSeconds: 300 (5 minutes)
  - Pod stays on the NotReady node for 5 minutes (hoping node recovers)

Phase 4: EVICTION (after 5 minutes)
  - If node doesn't recover, Pod is evicted
  - Pod object updated: status = Failed, reason = "NodeLost"
  - If Pod is managed by a controller (Deployment, StatefulSet):
      Controller sees fewer Pods than desired → creates replacement Pod
      Scheduler places new Pod on a healthy node

Phase 5: STUCK PODS (StatefulSet edge case)
  - StatefulSet won't create db-1 replacement until old db-1 is confirmed dead
  - If node is partitioned (not truly dead), old db-1 might still be running
  - Force delete may be needed: kubectl delete pod db-1 --force --grace-period=0
  - Risk: two copies of db-1 running (split-brain) — be careful with stateful
workloads

Timeline:
  T+0s:    Last heartbeat received
  T+40s:   Node marked NotReady
  T+40s:   NoExecute taint applied
  T+340s:  (5 min later) Pods evicted
  T+340s+: Replacement Pods scheduled elsewhere
```

Q23: How do resource requests and limits work? What happens when a Pod exceeds its limits?

Answer:

```
Requests: GUARANTEED minimum resources — used by scheduler for placement
Limits:   MAXIMUM resources — enforced by kubelet/cgroups

resources:
  requests:
    cpu: "250m"       ← scheduler ensures node has 250 millicores free
    memory: "256Mi"   ← scheduler ensures node has 256Mi free
  limits:
    cpu: "1000m"      ← can burst up to 1 core
    memory: "512Mi"   ← hard cap at 512Mi

CPU behavior:
  - Requests: Pod guaranteed 250m CPU time
  - If pod tries to use more than 1000m (limit): CPU THROTTLED
  - Pod is NOT killed for exceeding CPU limit — just slowed down
  - CPU is compressible — can be taken away and given back

Memory behavior:
  - Requests: 256Mi guaranteed
  - If Pod tries to allocate more than 512Mi (limit): OOMKilled
  - Memory is incompressible — can't "take back" allocated memory
  - kubelet kills the container, restarts it (CrashLoopBackOff if repeated)

QoS Classes (based on requests/limits):

  Guaranteed: requests == limits for ALL containers
    → Last to be evicted under node pressure
    → Best performance guarantees

  Burstable: requests < limits (or only requests set)
    → Evicted after BestEffort, before Guaranteed

  BestEffort: no requests or limits set
    → First to be evicted under node pressure
    → Avoid in production

Node under memory pressure:
  Phase 1: kubelet detects memory pressure
  Phase 2: Evicts BestEffort Pods first
  Phase 3: Evicts Burstable Pods exceeding requests
  Phase 4: Evicts Guaranteed Pods (last resort)
```

Q24: Explain Init Containers. When would you use them?

Answer:

```
Init containers run BEFORE app containers start, one at a time, sequentially.
All init containers must succeed before the app container starts.

Use cases:
  1. Wait for a dependency (DB, API) to be available
  2. Clone a git repo or download config before app starts
  3. Run database migrations
  4. Set file permissions on mounted volumes
  5. Register with a service registry

Example: Wait for database before starting app

  initContainers:
    - name: wait-for-db
      image: busybox
      command: ['sh', '-c',
        'until nc -z postgres-service 5432; do echo waiting; sleep 2; done']

  containers:
    - name: my-app
      image: my-app:v1

Flow:
  Phase 1: Pod scheduled, init container "wait-for-db" starts
  Phase 2: Init container polls postgres:5432 every 2s
  Phase 3: DB becomes available → init container exits successfully
  Phase 4: App container "my-app" starts
  Phase 5: Pod status: Running

If init container fails:
  → kubelet retarts it (based on Pod restartPolicy)
  → App container NEVER starts until init succeeds
  → Pod status: Init:Error or Init:CrashLoopBackOff
```

Q25: What are Custom Resource Definitions (CRDs) and Operators?

Answer:

```
CRD (Custom Resource Definition):
  - Extends the Kubernetes API with your own resource types
  - After creating a CRD, you can kubectl apply your custom objects
  - Stored in etcd just like Pods and Deployments

Example: Create a Database CRD
  apiVersion: apiextensions.k8s.io/v1
  kind: CustomResourceDefinition
  metadata:
    name: databases.mycompany.io
  spec:
    group: mycompany.io
    names:
      kind: Database
      plural: databases
    scope: Namespaced
    versions:
      - name: v1
        served: true
        storage: true

Now you can do:
  kubectl get databases
  kubectl apply -f my-postgres.yaml
    apiVersion: mycompany.io/v1
    kind: Database
    metadata:
      name: my-postgres
    spec:
      engine: postgresql
      version: "15"
      replicas: 3
      storage: 100Gi

But the CRD alone does nothing. You need...

OPERATOR:
  - A controller that watches for your custom resources and ACTS on them
  - Encodes operational knowledge (install, upgrade, backup, failover)
  - Runs as a Pod in the cluster

  Operator watches Database CRD:
    Phase 1: Sees "my-postgres" created
    Phase 2: Creates StatefulSet (3 replicas), Service, PVCs, ConfigMaps
    Phase 3: Configures replication (primary + 2 replicas)
```

```
    Phase 4: Sets up automated backups (CronJob)
    Phase 5: Monitors health, handles failover automatically

  Popular operators:
    - Prometheus Operator (monitoring)
    - Strimzi (Kafka on K8s)
    - Zalando Postgres Operator
    - Cert-Manager (TLS certificates)

CRD + Operator = extending Kubernetes to manage ANY stateful system
with the same declarative model (kubectl apply, reconciliation loops).
```

Q26: A Pod is in CrashLoopBackOff. How do you debug it?

Answer:

```
CrashLoopBackOff means: container starts, crashes, K8s restarts it,
crashes again, K8s backs off (increasing delays: 10s, 20s, 40s... up to 5min).

Debugging flow:

Step 1: CHECK POD EVENTS
  kubectl describe pod <pod-name>
  → Look at Events section:
    "Back-off restarting failed container"
    "OOMKilled"  ← memory limit exceeded
    "Error: ImagePullBackOff" ← wrong image (different issue)
    "CrashLoopBackOff" with exit code

Step 2: CHECK CONTAINER LOGS
  kubectl logs <pod-name>                  ← current container logs
  kubectl logs <pod-name> --previous       ← logs from PREVIOUS crashed
container
  (--previous is critical — current container may have no logs yet)

Step 3: CHECK EXIT CODE
  kubectl get pod <pod-name> -o
jsonpath='{.status.containerStatuses[0].lastState}'

  Exit code 0:    app exited normally (shouldn't happen for a server)
  Exit code 1:    app error (check logs for exception/stack trace)
  Exit code 137:  SIGKILL — likely OOMKilled (check memory limits)
  Exit code 139:  Segfault
```

```
  Exit code 143:  SIGTERM — graceful shutdown (but why?)

Step 4: CHECK RESOURCE LIMITS
  kubectl describe pod → look for "OOMKilled" in lastState
  If OOMKilled: increase memory limits

Step 5: RUN A DEBUG SHELL
  kubectl run debug --image=busybox -it --rm -- sh
  → Test connectivity, DNS, environment from inside the cluster

  Or ephemeral debug container (K8s 1.23+):
  kubectl debug -it <pod-name> --image=busybox --target=<container-name>

Common causes:
  - App crashes on startup (missing config, wrong env vars, bad connection
string)
  - OOMKilled (memory limit too low)
  - Missing dependency (DB not reachable, Secret not mounted)
  - Liveness probe misconfigured (kills healthy container)
  - Read-only filesystem but app tries to write temp files
```

Q27: How does Kubernetes handle rolling back a failed deployment?

Answer:

```
Kubernetes keeps old ReplicaSets as revision history.

Scenario: v2 deployment is failing (Pods crash on startup)

Automatic detection:
  - Rolling update creates v2 Pods
  - v2 Pods fail readiness probes
  - With maxUnavailable: 0, old v1 Pods are NOT killed
  - Rollout stalls — v2 Pods keep crashing, v1 Pods keep serving
  - progressDeadlineSeconds (default 600s) → after 10 min, Deployment
    condition "Progressing" = False

Manual rollback:
  kubectl rollout undo deployment/my-app
  → Reverts to previous ReplicaSet (v1)
  → v2 ReplicaSet scaled to 0
  → v1 ReplicaSet scaled back up
```

```
  → Traffic returns to v1 Pods

Rollback to specific revision:
  kubectl rollout history deployment/my-app
    REVISION  CHANGE-CAUSE
    1         initial deploy
    2         image update to v2
    3         image update to v3

  kubectl rollout undo deployment/my-app --to-revision=1

Under the hood:
  - Deployment controller copies the Pod template from the old ReplicaSet
  - Creates a new ReplicaSet with that template
  - Scales it up, scales current (broken) one down
  - It's effectively a new rollout to the old version

Prevention:
  - Use readiness probes (rollout won't proceed if new Pods aren't ready)
  - Set maxUnavailable: 0 (never kill old before new is ready)
  - Set progressDeadlineSeconds to catch stalled rollouts
```

Q28: Explain the difference between a Job and a CronJob.

Answer:

```
Job:
  - Runs one or more Pods to completion (exit code 0)
  - Pod is NOT restarted after successful completion
  - Good for: DB migration, batch processing, one-time tasks

  spec:
    completions: 5        ← run 5 Pods total (all must succeed)
    parallelism: 2        ← run 2 at a time
    backoffLimit: 3       ← retry failed Pods 3 times before marking Job as
failed
    activeDeadlineSeconds: 300  ← kill job if not done in 5 minutes

CronJob:
  - Creates Jobs on a schedule (cron syntax)
  - Good for: periodic cleanup, report generation, backups

  spec:
```

```
    schedule: "0 2 * * *"              ← every day at 2 AM
    concurrencyPolicy: Forbid          ← don't start new if previous still
running
    successfulJobsHistoryLimit: 3      ← keep 3 completed Jobs
    failedJobsHistoryLimit: 1          ← keep 1 failed Job
    startingDeadlineSeconds: 200       ← if missed by 200s, skip this run

  Concurrency policies:
    Allow:   run concurrent Jobs (default)
    Forbid:  skip new run if previous not finished
    Replace: kill previous, start new
```

Q29: Your production cluster suddenly has Pods stuck in Pending state. Walk through your troubleshooting.

Answer:

```
Pending means the scheduler cannot place the Pod on any node.

Debugging flow:

Step 1: DESCRIBE THE POD
  kubectl describe pod <pending-pod>
  → Check Events section for scheduler messages:

  Common messages and causes:

  "Insufficient cpu" / "Insufficient memory"
    → No node has enough free resources
    → Fix: scale down other workloads, add nodes, adjust requests

  "0/5 nodes are available: 5 node(s) had taint
   {node.kubernetes.io/not-ready: }, that the pod didn't tolerate"
    → All nodes are NotReady or have taints the Pod doesn't tolerate
    → Fix: check node health, add tolerations to Pod

  "0/5 nodes are available: 2 node(s) didn't match Pod's
   node affinity/selector"
    → Pod requires specific nodes (label, zone) that don't exist or are full
    → Fix: check nodeSelector/nodeAffinity, add matching nodes

  "persistentvolumeclaim "data" not found"
    → PVC doesn't exist or isn't bound
```

```
    → Fix: create PVC, check StorageClass, check PV availability

  "no persistent volumes available for this claim and
   no storage class is set"
    → Dynamic provisioning isn't configured
    → Fix: create StorageClass or manually create PV

Step 2: CHECK NODE RESOURCES
  kubectl describe nodes | grep -A 5 "Allocated resources"
  → See how much CPU/memory is allocated vs capacity
  → If allocated ≈ capacity: cluster is full

Step 3: CHECK CLUSTER AUTOSCALER LOGS (if using)
  kubectl logs -n kube-system -l app=cluster-autoscaler
  → Is it trying to scale up?
  → Is it hitting max node count?
  → Are there cloud API errors?

Step 4: CHECK FOR PDB BLOCKING
  kubectl get pdb
  → PDB might prevent evictions needed for rescheduling

Step 5: CHECK FOR RESOURCE QUOTAS
  kubectl describe quota -n <namespace>
  → Namespace might have hit its CPU/memory/Pod count quota
```

Q30: Design a production-ready Kubernetes deployment for a stateless microservice. What specs would you include and why?

Answer:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  labels:
    app: order-service
    version: v1
spec:
  replicas: 3                           # ← minimum for HA
  revisionHistoryLimit: 5                # ← keep 5 old ReplicaSets for rollback
  strategy:
```

```
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0                   # ← never kill old before new is ready
      maxSurge: 1                         # ← one extra Pod during rollout
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
        version: v1
    spec:
      serviceAccountName: order-service-sa  # ← least privilege SA, not default

      topologySpreadConstraints:            # ← spread across AZs
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: order-service

      containers:
        - name: order-service
          image: myregistry/order-service:v1.2.3  # ← pinned tag, never :latest
          ports:
            - containerPort: 8080

          resources:                        # ← always set both
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "1000m"
              memory: "512Mi"

          readinessProbe:                   # ← critical for zero-downtime
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            failureThreshold: 3
```

```
          livenessProbe:                    # ← detect deadlocked processes
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
            failureThreshold: 3

          startupProbe:                     # ← if app takes time to boot
            httpGet:
              path: /healthz
              port: 8080
            failureThreshold: 30
            periodSeconds: 2                # ← gives up to 60s to start

          lifecycle:
            preStop:                        # ← graceful shutdown race condition
fix
              exec:
                command: ["sleep", "5"]

          securityContext:                   # ← hardened
            runAsNonRoot: true
            runAsUser: 1000
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]

          env:
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: order-config
                  key: db_host
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: order-secrets
                  key: db_password

      terminationGracePeriodSeconds: 30     # ← time for graceful shutdown

---
apiVersion: policy/v1
```

```yaml
kind: PodDisruptionBudget                  # ← protect during disruptions
metadata:
  name: order-service-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: order-service

---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler              # ← auto-scale on CPU
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Why each piece matters:

- `maxUnavailable: 0` + readiness probe = zero-downtime deploys
- `preStop sleep` = handles kube-proxy iptables race condition
- `topologySpreadConstraints` = survive AZ failure
- `PDB` = safe node drains and cluster autoscaler scale-downs
- `securityContext` = defense in depth, non-root, read-only fs
- Pinned image tag = reproducible, no surprise `latest` pulls
- `HPA` = elastic scaling under load
- `resources` = proper scheduling + QoS class (Burstable)

Quick Reference: Commands You Should Know

```
# Cluster info
kubectl cluster-info
kubectl get nodes -o wide
kubectl top nodes / kubectl top pods

# Debugging
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl exec -it <pod> -- /bin/sh
kubectl debug -it <pod> --image=busybox

# Deployments
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl set image deployment/<name> container=image:tag

# Scaling
kubectl scale deployment/<name> --replicas=5
kubectl autoscale deployment/<name> --min=2 --max=10 --cpu-percent=60

# Resource inspection
kubectl get events --sort-by='.lastTimestamp'
kubectl get pods -o wide --all-namespaces
kubectl describe nodes | grep -A 10 "Allocated resources"

# RBAC
kubectl auth can-i create deployments --as=system:serviceaccount:staging:ci-bot -
n staging
kubectl auth can-i '*' '*' --as=<user>  # check admin access

# Drain / cordon
kubectl cordon <node>      # mark unschedulable
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>    # mark schedulable again
```